CSCI 4623/5623 **DIGITAL FORENSICS**

THE UNIVERSITY of NEW ORLEANS

03: STORAGE ANALYSIS

partitions, volumes, filesystems

Overview

- Partitions/volumes are units for “macro” storage management
  - split a single device into multiple (disjoint) management units
  - put together multiple physical partitions into a single logical one
- Filesystems (FS) are data structures for storing and organizing files
  - “micro” management
- Typical FS is designed for **stable** storage with a particular emphasis on consistency and durability
  - Exceptions: in-memory FS, specialized (pseudo) FS -- /proc, /dev, etc.
- Storage analysis
  - retrieval physical & logical data based on knowledge of the partition/FS data structures
  - emphasis on resilience and (incomplete) data retrieval

PRELIMINARIES

device naming conventions

HDD CONFIGURATION: SATA

- SATA (Serial ATA)
  - » the modern version of ATA
  - » serial → one data line, point-to-point connections
  - » throughput (ver 1, 2, 3): 1.5, 3, 6 Gb/s
  - » eSATA: → same spec + external connectivity
  - » BIOS will auto-detect but you may have to activate the device
  - » Linux treats the device as SCSI …

HDD CONFIGURATION: SCSI

- Small Computer System Interface ("scuzzy")
  - many versions, up to 640 MB/s
  - traditionally used on 'enterprise' devices (15k rpm)
  - SAS ➔ Serial Attached SCSI
  - HDD dev naming: /dev/sda, sdb, …
  - tape naming: `/dev/st0, st1`,…, or `/dev/nst0, nst1`
  - CD/DVD: `/dev/scd0, scd1`,…
- Notes:
  - SCSI device numbering is usually assigned based on SCSI device IDs.
  - Changes in the ID, the order in which drivers are loaded, or the introduction of new device could change the naming.

Ex: disk config

```
vassil@jeeves:/dev$ sudo fdisk -l sd[a-c]

Disk sda: 250.0 GB, 250000000000 bytes
255 heads, 63 sectors/track, 30394 cylinders
Units = cylinders of 16065 * 512 = 8225280 bytes
Disk identifier: 0xd0f4738c
Device Boot      Start         End      Blocks   Id  System
  sda1   *           1       29157   234203571   83  Linux
  sda2           29158       30394     9936202+   5  Extended
  sda5           29158       30394     9936171   82  Linux swap/Solaris

Disk sdb: 160.0 GB, 160000000000 bytes
255 heads, 63 sectors/track, 19452 cylinders
Units = cylinders of 16065 * 512 = 8225280 bytes
Disk identifier: 0x000d71b9
Device Boot      Start         End      Blocks   Id  System
  sdb1               1       19452   156248158+  83  Linux

Disk sdc: 750.2 GB, 750156374016 bytes
255 heads, 63 sectors/track, 91201 cylinders
Units = cylinders of 16065 * 512 = 8225280 bytes
Disk identifier: 0xa4b57300
Device Boot      Start         End      Blocks   Id  System
  sdc1               1       91201   732572001   83  Linux
```

HDD CONFIGURATION: EXTERNAL DRIVES

- USB/IEEE-1394 (FireWire) disks are treated as SCSI
  - As the device is plugged in, a corresponding `/dev/sd<x>` node appears in the file system.
  - NB: Make sure to unmount before unplugging!

Partitions/volumes

PARTITIONING: DEFINITION

- **Def:** The splitting of a block storage device into multiple, disjoint contiguous chunks of storage.
  - each chunk is known as a physical partition, or volume
- Conversely, physical partitions – on one, or more, devices – may be combined to present a unified contiguous view.
  - known as logical partitions/volumes

PARTITIONING: RATIONALE

- Management concerns
  - separation of use cases
    - e.g., boot partition, user directories, raw partitions, etc.
  - set usage & quota limits
  - backup
    - backing up a whole partition may be more efficient
- Performance/utilization
  - can choose different filesystems for each partition
- Reliability/fault tolerance
  - guard against single point of failure
- Security
  - allow for different mounting permissions

Common Linux partitions & mount points

- Mount point
  - a directory through which the partition is accessed
- Common partitions

/	root (must have it)
/home	user data
/usr	common system files (executables & data)
/usr/local	system files unique to installation
/opt	third-party packages
/var	mostly transient/data files; **logs** are in /var/log
/tmp	temporary files
/mnt	not a true partition—root dir for mounting media & remote fs
/media	not a true partition—root for mounting removable media

Managing partitions and filesystems

`fdisk`	basic partition manipulation
`parted`	most commonly used (can create FS as well)
`gparted`	GUI-enhanced parted (GTK)
`kpartx`	GUI-enhanced parted (KDE)
`mkfs`	system files unique to installation
`mount`	mount FS
`umount`	unmount FS
`losetup`	loopback device mount setup
`/etc/fstab`	fs table—filesystems to be mounted
`dumpe2fs`	ext FS info

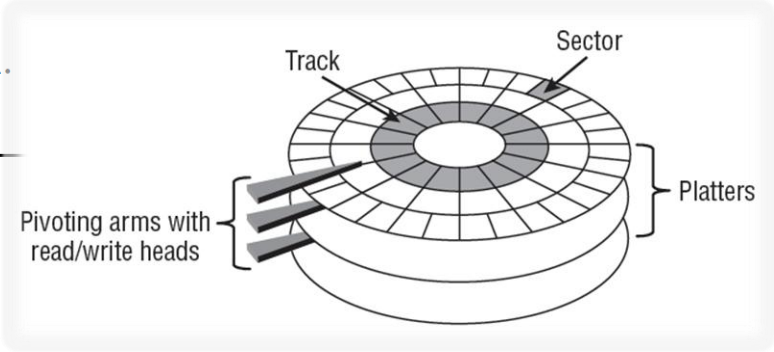
MSDOS PARTITIONS (LEGACY)

- Up to **4** ***primary*** partitions
- One of them can be an ***extended*** partition
  - can contain further ***logical*** partitions
- It is a baroque system developed to extend an originally limited system allowing only four partitions
- Linux SCSI/SATA device naming convention
  - /dev/sd**a**, /dev/sd**b**, /dev/sd**c**, …
  - legacy devices: hda, hdb, hdc, …; should not be around
  - primary partitions → indexes 1-4; e.g.:
    - /dev/sdb1, ..., /dev/sdb4
  - logical → indexes 5+:
    - /dev/sdb5, /dev/sdb6, …
    - this is true even if less than four primary partitions are present

MSDOS PARTITIONS

- Master boot record contains a table with 4 entries of the form:

Offset	Length	Description
00	1	Status: `0x80` = active, `0x00` = inactive.
01	3	CHS address of the *first* sector in the partition.
04	1	Partition type
05	3	CHS address of the *last* sector in the partition.
08	4	LBA address of the *first* sector in the partition.
12	4	Number of sectors in the partition.
	16	Total length



- CHS *cylinder-head-sector* addressing (legacy)
  - » 63 sec/track x 255 heads (tr/cyl) x 1024 cylinders x 512 B/sec = 8032.5MiB
  - » known as 8GB limit

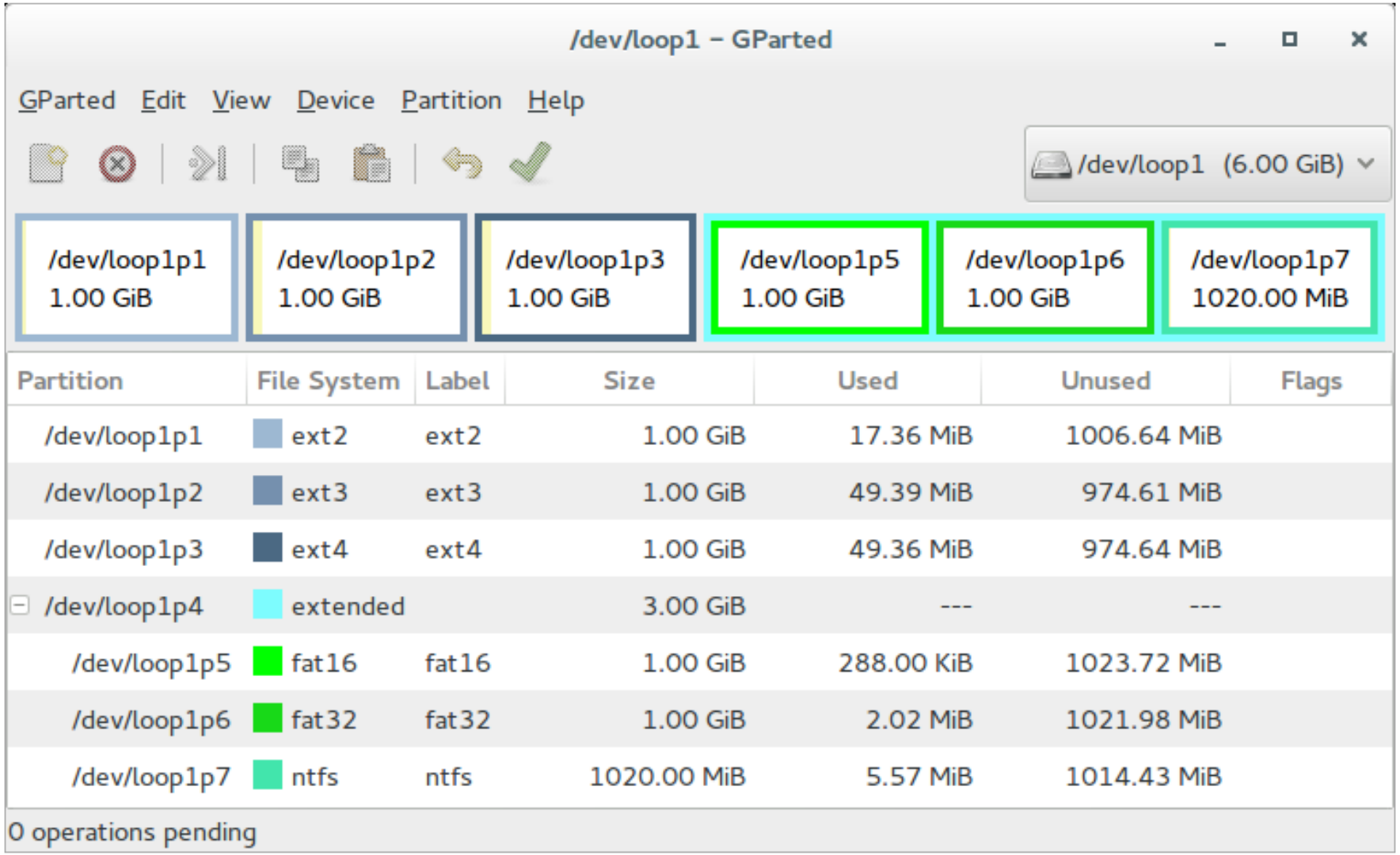
PARTITION CODES

- A one-byte identifier of partition type (OS-dependent). E.g.:
  - `0x05` DOS 3.3+ extended partition
  - `0x06` FAT32
  - `0x07` NTFS
  - `0x0e` WIN95: DOS 16-bit FAT, LBA-mapped
  - `0x0f` WIN95: Extended partition, LBA-mapped
  - `0x82` Linux swap
  - `0x83` Linux native filesystem
- Detailed list

EXAMPLE #1: 64M DEVICE W/ TWO PARTITIONS

```
# mmls -B target-64M.dd
DOS Partition Table
Offset Sector: 0
Units are in 512-byte sectors

     Slot     Start        End          Length       Size    Description
00:  Meta     0000000000   0000000000   0000000001   0512B   Primary Table (#0)
01:  -----    0000000000   0000002047   0000002048   1024K   Unallocated
02:  00:00    0000002048   0000083967   0000081920   0040M   Win95 FAT32 (0x0B)
03:  00:01    0000083968   0000131071   0000047104   0023M   Linux (0x83)
```

Example #2: 3 primary + 3 logical partitions (gparted)



Partition	File System	Label	Size	Used	Unused	Flags
/dev/loop1p1	ext2	ext2	1.00 GiB	17.36 MiB	1006.64 MiB	
/dev/loop1p2	ext3	ext3	1.00 GiB	49.39 MiB	974.61 MiB	
/dev/loop1p3	ext4	ext4	1.00 GiB	49.36 MiB	974.64 MiB	
/dev/loop1p4	extended		3.00 GiB	---	---	
/dev/loop1p5	fat16	fat16	1.00 GiB	288.00 KiB	1023.72 MiB	
/dev/loop1p6	fat32	fat32	1.00 GiB	2.02 MiB	1021.98 MiB	
/dev/loop1p7	ntfs	ntfs	1020.00 MiB	5.57 MiB	1014.43 MiB	

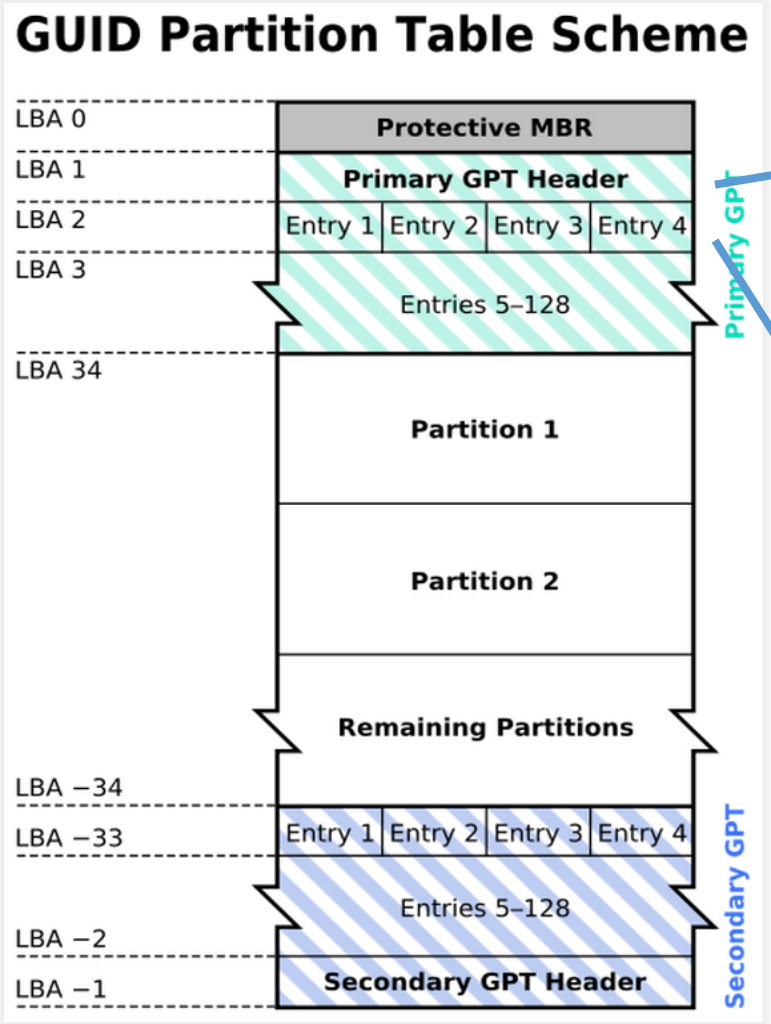
Example #2: 3 primary + 3 logical partitions (mmls)

```
     Slot   Start       End         Length      Size  Description
00:  Meta   0000000000  0000000000  0000000001  0512B Primary Table (#0)
01:  -----  0000000000  0000002047  0000002048  1024K Unallocated
02:  00:00  0000002048  0002099199  0002097152  1024M Linux (0x83)
03:  00:01  0002099200  0004196351  0002097152  1024M Linux (0x83)
04:  00:02  0004196352  0006293503  0002097152  1024M Linux (0x83)
05:  Meta   0006293504  0012582911  0006289408  0002G DOS Extended (0x05)
06:  Meta   0006293504  0006293504  0000000001  0512B Extended Table (#1)
07:  -----  0006293504  0006295551  0000002048  1024K Unallocated
08:  01:00  0006295552  0008392703  0002097152  1024M DOS FAT16 (0x06)
09:  Meta   0008392704  0010491903  0002099200  0001G DOS Extended (0x05)
10:  Meta   0008392704  0008392704  0000000001  0512B Extended Table (#2)
11:  -----  0008392704  0008394751  0000002048  1024K Unallocated
12:  02:00  0008394752  0010491903  0002097152  1024M Win95 FAT32 (0x0B)
13:  Meta   0010491904  0012582911  0002091008  1021M DOS Extended (0x05)
14:  Meta   0010491904  0010491904  0000000001  0512B Extended Table (#3)
15:  -----  0010491904  0010493951  0000002048  1024K Unallocated
16:  03:00  0010493952  0012582911  0002088960  1020M NTFS (0x07)
```

/ETC/FSTAB → THE MOUNT TABLE

```
# Use 'blkid -o value -s UUID' to print the universally unique identifier
# for a device; this may be used with UUID= as a more robust way to name
# devices that works even if disks are added and removed. See fstab(5).
#
# <file system> <mount point>   <type>     <options>      <dump>  <pass>
proc <tab>      /proc <tab>     proc <tab> nodev,noexec,nosuid <tab>   0 <tab>  0

# / was on /dev/sda1 during installation
UUID=a2dfece4-c06b-419d-b869-2d412660c2cb  /    ext4   errors=remount-ro  0    1

# swap was on /dev/sda5 during installation
UUID=14db7f30-a2ff-4755-b43e-63f29cd713b5  none  swap  sw                 0    0

# home → moved subsequently
UUID=432726c4-d642-4a9b-8b71-5a1e0fbaa092  /home ext4  errors=remount-ro  0    1

/dev/fd0        /media/floppy0  auto    rw,user,noauto,exec,utf8          0    0
```

GUID Partition Table (GPT)



GPT header format

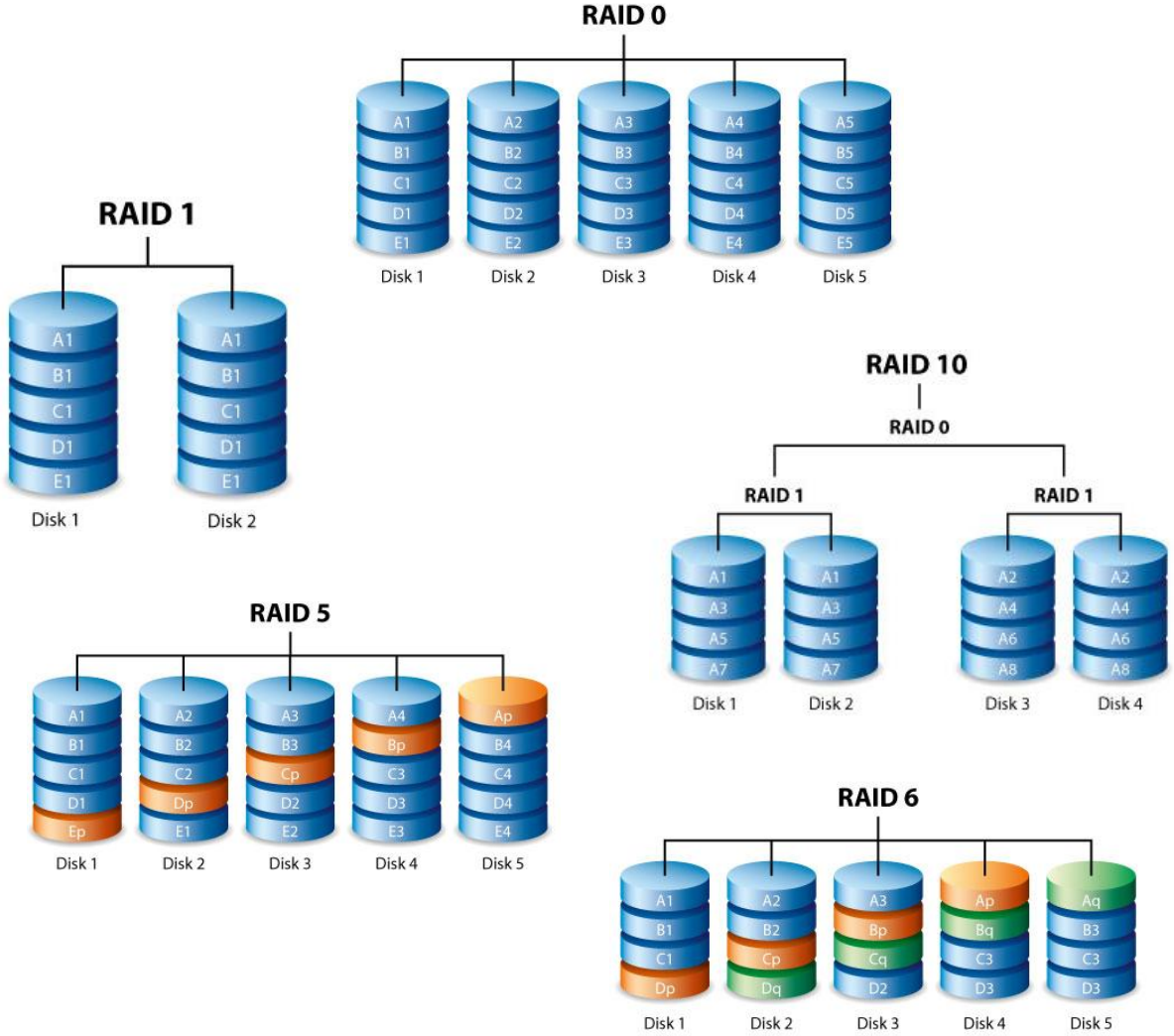
Offset	Length	Contents
+0	8 bytes	Signature ("EFI PART", `45h 46h 49h 20h 50h 41h 52h 54h`)
+8	4 bytes	Revision (for GPT version 1.0 (through at least UEFI version 2.3.1), the value is `00h 00h 01h 00h`)
+12	4 bytes	Header size in little endian (in bytes, usually `5Ch 00h 00h 00h` meaning 92 bytes)
+16	4 bytes	CRC32 of header (offset +0 up to header size), with this field zeroed during calculation
+20	4 bytes	Reserved; must be zero
+24	8 bytes	Current LBA (location of this header copy)
+32	8 bytes	Backup LBA (location of the other header copy)
+40	8 bytes	First usable LBA for partitions (primary partition table last LBA + 1)
+48	8 bytes	Last usable LBA (secondary partition table first LBA - 1)
+56	16 bytes	Disk GUID (also referred as UUID on UNIXes)
+72	8 bytes	Starting LBA of array of partition entries (always 2 in primary copy)
+80	4 bytes	Number of partition entries in array
+84	4 bytes	Size of a single partition entry (usually 128)
+88	4 bytes	CRC32 of partition array
+92	*	Reserved; must be zeroes for the rest of the block (420 bytes for a sector size of 512 bytes; but can be n
LBA size		**Total**

GUID partition entry format

Offset	Length	Contents
0	16 bytes	Partition type GUID
16	16 bytes	Unique partition GUID
32	8 bytes	First LBA (little endian)
40	8 bytes	Last LBA (inclusive, usually odd)
48	8 bytes	Attribute flags (e.g. bit 60 denotes read-only)
56	72 bytes	Partition name (36 UTF-16LE code units)
128		**Total**

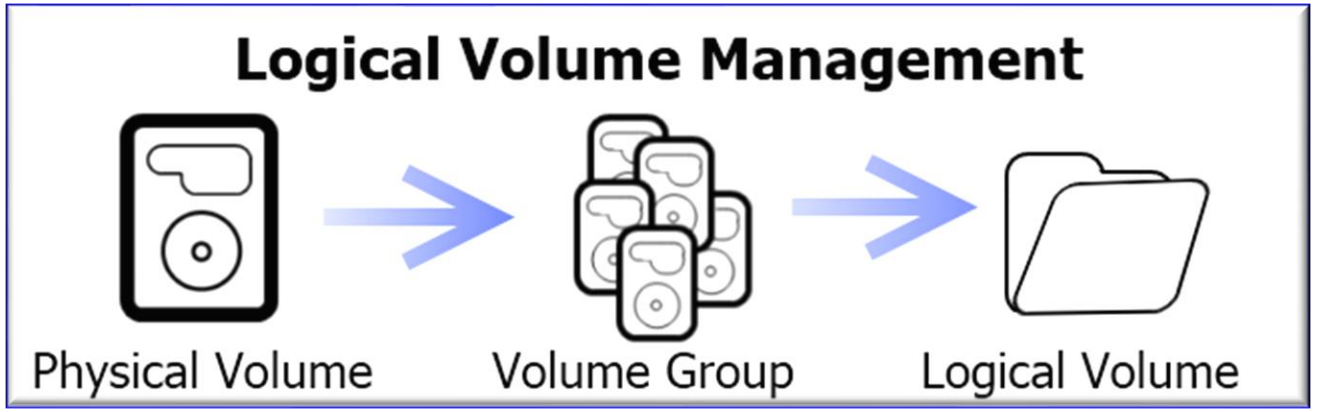
Quick RAID review

- RAID 0
  - » striping
  - » no overhead
- RAID 1
  - » mirroring
  - » 50% overhead
- RAID 10 (1+0)
  - » combines 1 and 0
- RAID 5
  - » one parity device
  - » usually 20-25% overhead
- RAID 6
  - » two parity devices
- Also, RAID 50, 60

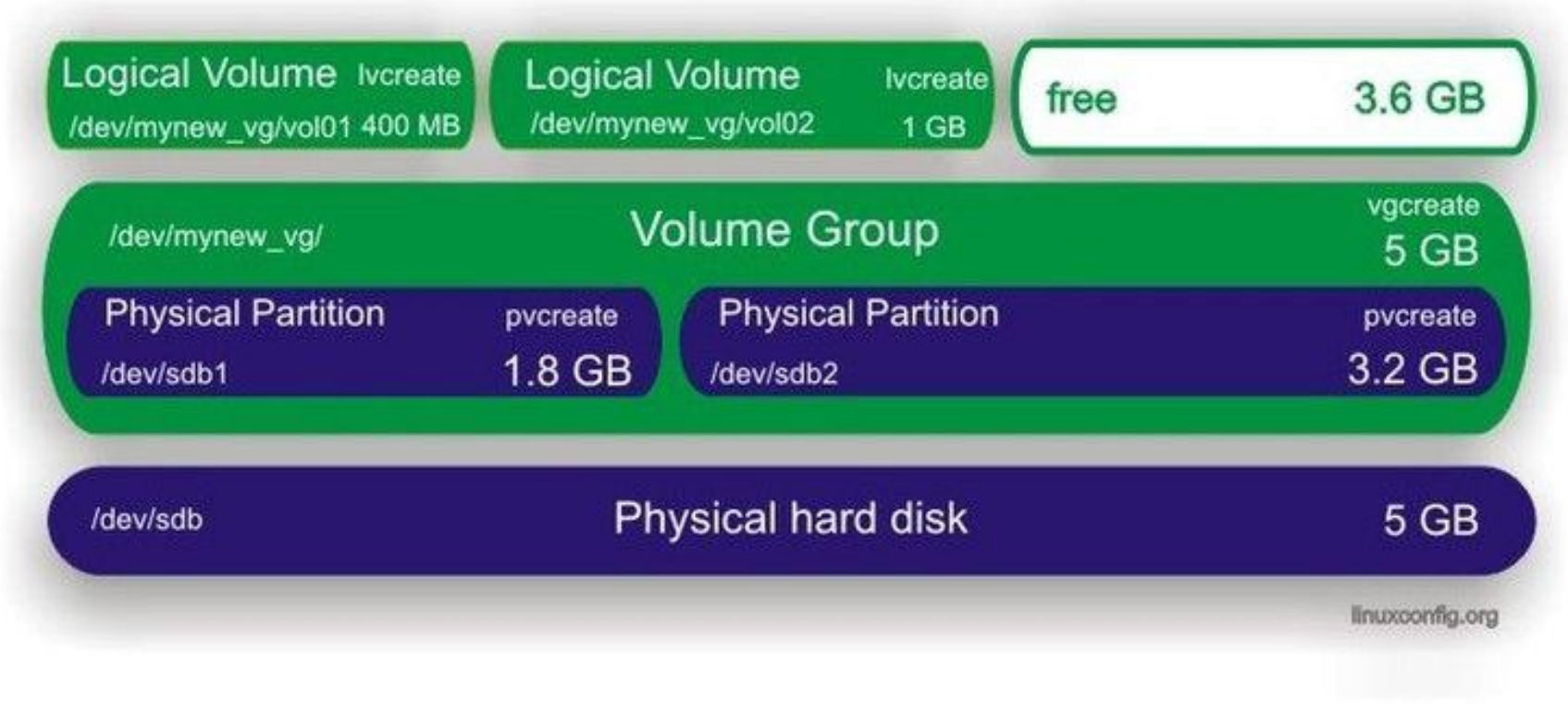


Quick LVM review

- Allows flexible management of multiple disks/partitions
  - Resize partitions as needed
  - Manage disk expansions transparently
  - Snapshot-based backups
  - Create single volume out of multiple physical ones (a la JBOD)



EXAMPLE **LVM** CONFIG



[Online LVM tutorial]

Exercise

file partition manipulation

Task #1: Create a loopback block device

- Loopback device is backed by a file; create one:
  - » `dd bs=1M count=256 if=/dev/zero of=256M.zero`
- Let's clone it, twice:
  - » `dd if=256M.zero of=256M-msdos.dd` # for **MSDOS** partitions
  - » `dd if=256M.zero of=256M-gpt.dd` # for **GPT** partitions
- Setup as loopback devices:

```
# losetup /dev/loop0 256-msdos.dd
# losetup /dev/loop0 256-gpt.dd
```

Task #2: Create partitions & filesystems

- Use `parted`, or `fdisk`, to create the following primary partitions (on each):
  - **P1** → 64MiB fat32
  - **P2** → 64 MiB ntfs
  - **P3** → remainder (~127MiB) ext4
- You should see the following devices in /dev
  - `loop0, loop0p1, loop0p2, loop0p3`
  - `loop1, loop1p1, loop1p2, loop1p3`

Task #3: Mount the partitions

- Create the following empty directory structure in your working folder:

```
msdos
msdos/p1
msdos/p2
msdos/p3
gpt
gpt/p1
gpt/p2
gpt/p3
```

- Mount the corresponding partitions

```
# for p in p1 p2 p3; do
    mount /dev/loop0${p} msdos/$p
    mount /dev/loop1${p} gpt/$p
done
```

Task #4 Create content & swap partitions

- Create a few files on the `p1` partitions
  - » different on each mount point
- Unmount all partitions
- Using dd:
  - » copy the **`loop0p1`** and **`loop1p1`** partitions to a temp file
  - » overwrite the **`loop0p1`** partition with the copy of **`loop1p1`**
  - » overwrite the **`loop1p1`** partition with the copy of **`loop0p1`**
- Remount the partitions as before
- What did you expect the result to be?
- Does the result match up with your expectations?

Exercise

quantifying the effects of FS format operations

PREREQUISITE

- `dcfldd`
  - a version of `dd` with some essential forensic functions added
- Relevant options
  - `hash`
  - `hashwindow`
  - `hashlog`
  - `totalhashformat`

Scenario [1]

- Create a blank 256M loopback device
  - see prior exercise
- Create a single partition
- Using `dcfldd`, create a copy (aka *image*) and store all the 4K block hashes
  - call this **`blank.dd`**
- Format the partition as FAT32
- Create an image with `dcfldd`, call it `fat32.dd`
  - store all the 4K hashes

Scenario [2]

- Format the partition as NTFS
  - **`mkfs.ntfs`**
- Create an image with `dcfldd`, call it `ntfs.dd`
  - store all the 4K hashes in `ntfs.hash`
- Format the partition as ext4
  - **`mkfs.ext4`**
- Create an image with `dcfldd`, call it `ext4.dd`
  - store all the 4K hashes in **`ext4.hash`**
- Quantify the differences in 4K blocks between
  - **`blank.dd`** and each of the subsequent images
  - **`fat32.dd`** and each of the subsequent images
  - **`ntfs.dd`** and **`ext4.dd`**

Scenario [3]

- Repeat the exercise with the following modification
  - after formatting with **FAT32**, and *before* you take an image:
    - mount the partition
    - fill it up *completely* with files
    - unmount it
- Q: Do the relative results differ from the base case?

EXT4 (2)

- Persistent pre-allocation for disk files
- Delayed space allocation
- Large # of subdirectories supported
  - 64,000 instead of 32,000 in ext3
- Journal checksumming
- Faster file system checking
  - unallocated block groups and inode table sections are marked and skipped
- Improved timestamps
  - **nanosecond** granularity